

Week 2

Session 6: Functional Programming using map()

1 Shuffle the Array –

Java code:

```
class Solution {  
    public int[] shuffle(int[] nums, int n) {  
  
        int[] result = new int[2 * n];  
  
        int index = 0;  
  
        for (int i = 0; i < n; i++) {  
  
            result[index] = nums[i];  
            index++;  
  
            result[index] = nums[i + n];  
            index++;  
        }  
  
        return result;  
    }  
}
```

Output:

nums =[2,5,1,3,4,7]

n =3

Session 7:

Functional Programming using filter():

1. Remove Element –

Java code:

```
class Solution {  
    public int removeElement(int[] nums, int val) {  
  
        int k = 0;  
  
        for (int i = 0; i < nums.length; i++) {  
  
            if (nums[i] != val) {  
                nums[k] = nums[i];  
                k++;  
            }  
        }  
  
        return k;  
    }  
}
```

Output:

nums =[3,2,2,3]

val =3

2. Remove Duplicates from Sorted Array –

Java code:

```
class Solution {  
    public int removeDuplicates(int[] nums) {  
  
        int k = 1;  
  
        for (int i = 1; i < nums.length; i++) {  
  
            if (nums[i] != nums[i - 1]) {  
                nums[k] = nums[i];  
                k++;  
            }  
        }  
  
        return k;  
    }  
}
```

Output:

nums =[1,1,2]

Output=[1,2]

Session 8

Functional Programming using reduce():

1. Maximum Subarray –

Java code:

```
class Solution {  
    public int maxSubArray(int[] nums) {  
  
        int currentSum = nums[0];  
        int maxSum = nums[0];  
  
        for (int i = 1; i < nums.length; i++) {  
  
            currentSum = Math.max(nums[i], currentSum + nums[i]);  
  
            maxSum = Math.max(maxSum, currentSum);  
        } return maxSum;  
    }  
}
```

Output:

nums = [-2,1,-3,4,-1,2,1,-5,4]

6

2. Find the Highest Altitude –

Java code:

```
class Solution {  
    public int largestAltitude(int[] gain) {  
  
        int altitude = 0;  
        int maxAltitude = 0;  
  
        for (int i = 0; i < gain.length; i++) {  
            altitude = altitude + gain[i];  
  
            if (altitude > maxAltitude) {  
                maxAltitude = altitude;  
            }  
        }  
  
        return maxAltitude;  
    }  
}
```

Output:

gain =[-5,1,5,0,-7]

1

Session 9

Stream & Pipeline Processing

1. Group Anagrams –

Java code:

```
import java.util.*;

class Solution {

    public List<List<String>> groupAnagrams(String[] strs) {

        HashMap<String, List<String>> map = new HashMap<>();

        for (String str : strs) {

            char[] chars = str.toCharArray();

            Arrays.sort(chars);

            String key = new String(chars);

            if (!map.containsKey(key)) {

                map.put(key, new ArrayList<>());

            } map.get(key).add(str);

        } return new ArrayList<>(map.values());

    }

}
```

Output:

```
[["eat","tea","ate"],["bat"],["tan","nat"]]
```

2. Top K Frequent Elements –

Java code:

```
import java.util.*;

class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        // Count frequency
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int num : nums) {
            map.put(num, map.getDefault(num, 0) + 1);
        } // Bucket: index = frequency
        List<Integer>[] buckets = new List[nums.length + 1];
        for (int num : map.keySet()) {
            int frequency = map.get(num);
            if (buckets[frequency] == null) {
                buckets[frequency] = new ArrayList<>();
            }
            buckets[frequency].add(num);
        }
        // Get highest frequencies
        int[] result = new int[k];
        int index = 0;
        for (int frequency = buckets.length - 1;
            frequency >= 0 && index < k;
            frequency--) {
            if (buckets[frequency] != null) {

                for (int num : buckets[frequency]) {
```

```
        result[index] = num;
        index++;
    if (index == k) {
        break;
    }
}
}
}
}
return result;
}
}
```

Output:

nums =[1,1,1,2,2,3]

k =2

Session 10 :
Stream API & Collection Processing

1. Java Dequeue –

Java code:

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int m = sc.nextInt();

        Deque<Integer> deque = new ArrayDeque<>();

        HashMap<Integer, Integer> map = new HashMap<>();

        int maxUnique = 0;

        for (int i = 0; i < n; i++) {

            int num = sc.nextInt();

            deque.addLast(num);

            map.put(num, map.getOrDefault(num, 0) + 1);

            // Window size m ayyaka

            if (deque.size() == m) {

                // Unique count

                maxUnique = Math.max(maxUnique, map.size());

                // First element remove

                int removed = deque.removeFirst();

                map.put(removed, map.get(removed) - 1);

                // Frequency 0 ayithe map nunchi remove
```

```
        if (map.get(removed) == 0) {  
            map.remove(removed);  
        }  
    }  
}  
System.out.println(maxUnique);  
}
```

Sample Input

6 3

5 3 5 2 3 2

Sample Output

3

2. Java Hashset –

Java code:

```
Scanner sc = new Scanner(System.in);

int n = sc.nextInt();

HashSet<String> set = new HashSet<>();

for (int i = 0; i < n; i++) {

    String first = sc.next();
    String second = sc.next();

    String pair = first + "#" + second;

    set.add(pair);

    System.out.println(set.size());
}
```

Sample Input

5

john tom

john mary

john tom

mary anna

mary anna

VTU29104

Sample Output

1

2

2

3

3